



FACULTY OF COMPUTER SCIENCE AND MATHEMATICS

CSE3433 - SOFTWARE ARCHITECTURE

SEMESTER II SESSION 25/26

BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING)

WITH HONOURS

Logbook

Submitted To:

Dr. Mohamad Nor Hasan

NAME	MATRIC NO.
SATRIALDY DARREL FARIZKI	S76351

Logbook

Week / Date	Development Phase	Key Tasks Completed	Technical Notes & Resolutions
Week 1 May 25 – May 31	Infrastructure & Architecture	Established the polyrepo architecture and global infrastructure. Configured drcp-infra Docker Compose containing PostgreSQL and Keycloak. Set up the shared external Docker network (drdp_network) to bridge isolated repositories.	Resolved a critical Docker database initialization conflict by separating POSTGRES_DB defaults from the init-keycloak-db.sh script, ensuring clean multi-database creation.
Week 2 Jun 1 – Jun 7	Microservices & IAM Setup	Bootstrapped the Quarkus incident-service with REST, Panache, Flyway, and OIDC dependencies. Configured the Keycloak Admin Console, creating the drcp realm, confidential clients, and user roles (coordinator). Secured backend endpoints using @RolesAllowed.	Fixed a Docker build crash by implementing the required Maven compile step (./mvnw clean package) before containerization. Resolved Hibernate schema validation errors by implementing initial Flyway SQL migrations.

Week 2 Jun 9	Incident Service – Data Layer	Created Flyway schema migration for incident, incident_attachment, incident_status_history with geolocation, images, and status history. Built seed data file for realistic tests.	Switched primary keys from BIGSERIAL to UUID using gen_random_uuid(). Used PanacheRepositoryBase<Entity, UUID> pattern to match UUID keys. Schema validated against PostgreSQL.
Week 2 Jun 9	Incident Service – API	Designed IncidentResource following the UserAdminResource style (Quarkus REST + JAX-RS). Created DTOs (CreateIncidentDto, UpdateIncidentDto) as Java records with Jakarta validation. Implemented IncidentService and IncidentRepository using a generic BaseRepository<T>.	The generic BaseRepository avoids code duplication and works seamlessly with Panache. DTOs leverage Java records for immutability. Applied @RolesAllowed("USER") for security.
Week 2 Jun 10	CORS Troubleshooting	Diagnosed CORS "(null)" error resulting from missing FRONTEND_URL environment variable resolution.	Root cause: environment variable not loaded by Quarkus. Resolved by adding a default origin fallback in properties: quarkus.http.cors.origins=\${FRONTEND_URL:http://localhost:5173}.

Week 3 Jun 8 – Jun 14	Base Frontend UI & API Integration	Built the React 19 Single Page Application with TypeScript and React Router 7. Implemented the authentication flow using react-oidc-context with a silent SSO check strategy. Developed the AppShell.tsx component to guard protected routes.	Configured an Axios global interceptor to seamlessly check token expiration and dynamically inject the Keycloak JWT Bearer token into all outgoing backend API requests.
Week 3 Jun 10	Order Feature – Architecture	Designed unified order model for dine-in and reservation flows (single orders table). Defined clear status lifecycle and role permissions (Manager, Staff, Customer). Created Flyway migration for tables, menu_items, reservations, orders, order_items.	Unified model avoids duplicate logic; only the creation path differs. Implemented composite index (type, status) for efficient dashboard filtering.

Week 3 Jun 10	Order Feature – Repository	Built OrderRepository extending BaseRepository<Order>. Implemented searchOrders using HQL and pagination, plus filtered lookups by status, type, customer, table, and date range.	Utilized Panache's PanacheQuery for seamless list + count execution in one database call.
Week 3 Jun 10	Order Feature – Resource	Built OrderResource mimicking the InventoryResource pattern (CRUD + status movements). Created DTO records: OrderResponse, CreateOrderRequest, StatusChangeRequest. Designed OrderStatusHistory entity and Flyway migration for audit trail.	Status history architecture mirrors the incident status tracking pattern for consistency.
Week 3 Jun 10	Order Feature – Frontend	Converted monolithic staff list page into modular React components for the order index. Created useOrders hook, OrderFilters, OrderTable, and OrderPagination.	Components are highly reusable for future order-related views. Applied consistent token-based styling.

Week 3 Jun 10 – 13	Order Feature – Polish	Corrected seeder column names (price_at_order, custom_instructions) to match the actual order_items table schema. Added missing indexes for all foreign keys.	Ensured query performance under load for common query patterns on orders, order_items, and reservations.
Week 4 Jun 15 – Jun 19	System Documentation	Authored the complete final project report detailing the shift from a monolithic concept to a decoupled microservice architecture. Documented the Problem Description, System Requirements, and Technology Stack.	Ensured the project contribution records accurately reflect solo full-stack development, architectural design, and documentation execution.

Gen-AI Usage

Component	Description
Tool Used	Gemini
Purpose	Assisted in resolving complex Docker container startup crashes caused by overlapping database initialization commands across a polyrepo environment.
Prompt Used	"So for each of my services, I have made a init-db.sh to create the initial database needed for the project, but for some reason, when I tried to run the containers, the healthcheck returns an error because it said that the database with the name has already been found in the postgres container, now check my .env, application.properties, and docker-compose.yml to see if there's any duplicate database creation or something else"
Output Summary	Identified a race condition between Docker's native POSTGRES_DB environment variable and the custom init-db.sh script attempting to build the exact same schema simultaneously. Recommended two clear structural paths to isolate the databases correctly.
How It Was Used	Used to rewrite the docker-compose.yml and .env files, successfully decoupling the Identity Service database from the Keycloak IAM database without triggering container failures.

Component	Description
Tool Used	Gemini

Purpose	Assisted in validating the architectural approach for cross-service authentication without duplicating Keycloak instances across multiple repositories.
Prompt Used	"I think I'm going with the polyrepo method, since it's already like that, and for now, the keycloak is initialized from my identity-service, I'm not asking you to create a centralized infra, I just want to know if the keycloak can manage other services too, so login and register and everything through the identity-service, but will it be able to authenticate endpoints across multiple different services"
Output Summary	Confirmed the viability of a single Keycloak instance acting as a centralized Identity Provider (IdP) for distributed Resource Servers. Explained the mechanics of stateless JWT signature verification using asymmetric cryptography (public keys via jwks_uri).
How It Was Used	Used to configure the incident-service properties and establish the shared drcp_network Docker bridge, allowing the Incident API to securely authenticate incoming tokens without direct database access to the Identity Service.